SUPPORT DE TRAVAUX PRATIQUES

Mini-espace de données fédéré — Santé et qualité de l'air

Master IASD — Data Spaces & Data Integration & Semantic Interoperability

Présentation générale

Un espace de données régional consacré à la santé et à la qualité de l'air est en cours de constitution. Il doit permettre à plusieurs organisations d'étudier l'association entre la pollution atmosphérique et les admissions hospitalières pour affections respiratoires.

Trois acteurs participent au projet :

- **Organisation A** — Direction régionale de l'environnement : mesures quotidiennes de PM2.5.
- **Organisation B** — Hôpital régional : nombre quotidien d'admissions respiratoires.
- **Organisation C** — Laboratoire universitaire : analyse des résultats autorisés.

Les quatre TP construisent le prototype étape par étape :

TP	Question traitée	Ce que l'on construit
TP 1	Où sont les données ?	Deux bases locales indépendantes
TP 2	Comment les organisations communiquent-elles ?	Deux API et un client (mais le laboratoire reçoit les données brutes)
TP 3	Comment éviter d'envoyer les données brutes ?	Calcul local chez l'hôpital, clé API
TP 4	Comment contrôler l'usage et limiter ce qui est révélé ?	Catalogue, contrat, signature, seuil minimal, bruit

Objectifs généraux

- Comprendre le stockage fédéré et la conservation des données chez leurs propriétaires.
- Créer et manipuler deux bases de données locales avec SQLite.
- Exposer des données et résultats par des API REST avec FastAPI.
- Interroger des services distincts depuis un client Python.
- Effectuer des calculs locaux et échanger des statistiques agrégées.
- Appliquer une gouvernance (finalité, contrat) et discuter des limites de la confidentialité.

Jeux de données fictifs (TP 1 à 3)

Fichier logique A — qualité de l'air :

Date	Ville	PM2.5 (µg/m³)
2026-10-01	Tanger	12
2026-10-02	Tanger	35
2026-10-03	Tanger	42
2026-10-04	Tanger	15
2026-10-05	Tanger	38

Fichier logique B — admissions hospitalières :

Date	Ville	Admissions respiratoires
2026-10-01	Tanger	8
2026-10-02	Tanger	14
2026-10-03	Tanger	18
2026-10-04	Tanger	9
2026-10-05	Tanger	16

Pour ces TP, une journée de pollution élevée est définie pédagogiquement par **PM2.5 > 30 µg/m³** .

TP 1 — Stockage fédéré et premières analyses locales

1. Objectifs

- Installer Python et Visual Studio Code.
- Créer deux bases SQLite distinctes.
- Insérer et consulter les données de chaque organisation.
- Comprendre pourquoi les données peuvent rester chez leurs propriétaires.

2. Outils à installer

- Python 3.10 ou plus : https://www.python.org/downloads/
- Visual Studio Code : https://code.visualstudio.com/Download

Dans VS Code, installez l'extension Python de Microsoft. SQLite est accessible via le module `sqlite3` intégré à Python ; aucun serveur SQLite séparé n'est requis.

3. Préparer le projet

Créez un dossier nommé `TP_Espace_Sante`. Dans VS Code, ouvrez ce dossier puis *Terminal > New Terminal*. Vérifiez Python :

```
py --version
# Si py n'est pas reconnu (ou sous Linux / macOS) :
python --version
python3 --version
```

La version doit être **3.10 ou supérieure**.

4. Créer les bases locales

Créez le fichier `tp_federation.py` et copiez le code suivant. Il crée `environnement.db` et `hopital.db` dans le dossier courant.

```
import sqlite3

# Organisation A : environnement
db_air = sqlite3.connect("environnement.db")
db_air.execute("""
CREATE TABLE IF NOT EXISTS qualite_air (
    date TEXT PRIMARY KEY,
    ville TEXT,
    pm25 REAL
)
""")
db_air.execute("DELETE FROM qualite_air")
donnees_air = [
    ("2026-10-01", "Tanger", 12),
    ("2026-10-02", "Tanger", 35),
    ("2026-10-03", "Tanger", 42),
    ("2026-10-04", "Tanger", 15),
    ("2026-10-05", "Tanger", 38)
]
db_air.executemany(
    "INSERT INTO qualite_air VALUES (?, ?, ?)", donnees_air
)
db_air.commit()
db_air.close()

# Organisation B : hôpital
db_hopital = sqlite3.connect("hopital.db")
db_hopital.execute("""
CREATE TABLE IF NOT EXISTS admissions (
    date TEXT PRIMARY KEY,
    ville TEXT,
    nombre INTEGER
)
""")
db_hopital.execute("DELETE FROM admissions")
donnees_hopital = [
    ("2026-10-01", "Tanger", 8),
    ("2026-10-02", "Tanger", 14),
```

```
    ("2026-10-03", "Tanger", 18),
    ("2026-10-04", "Tanger", 9),
    ("2026-10-05", "Tanger", 16)
]
db_hopital.executemany(
    "INSERT INTO admissions VALUES (?, ?, ?)", donnees_hopital
)
db_hopital.commit()
db_hopital.close()

print("Deux bases locales ont été créées.")
```

Exécutez le programme dans le terminal :

```
python tp_federation.py
```

Résultat attendu : « Deux bases locales ont été créées. » Deux fichiers apparaissent dans le dossier du projet : `environnement.db` et `hopital.db`.

5. Première consultation des bases

Créez `consulter_bases.py` pour afficher le contenu des deux bases :

```
import sqlite3

requetes = [
    ("environnement.db", "SELECT * FROM qualite_air ORDER BY date"),
    ("hopital.db", "SELECT * FROM admissions ORDER BY date"),
]

for fichier, requete in requetes:
    print("\nBase :", fichier)
    db = sqlite3.connect(fichier)
    for ligne in db.execute(requete):
        print(ligne)
    db.close()
```

Exécutez : `python consulter_bases.py`. Vous devez voir cinq lignes dans chaque base.

6. Questions TP 1

1. Où sont stockées les mesures de qualité de l'air ?
2. Où sont stockées les admissions hospitalières ?
3. Les deux jeux de données sont-ils dans une même base ?
4. Comment le laboratoire pourrait-il découvrir les données disponibles sans centraliser les fichiers ?

TP 2 — Communication entre organisations avec des API REST

1. Objectifs

- Installer FastAPI, Uvicorn et Requests.
- Exposer chaque base par une API distincte.
- Tester les API avec un navigateur et Swagger.
- Interroger les deux organisations depuis un client Python.

2. Installation

Dans le terminal, créez et activez un environnement virtuel.

Windows PowerShell :

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install fastapi uvicorn requests
```

Si Windows bloque l'activation (« l'exécution de scripts est désactivée »), tapez d'abord :

```
Set-ExecutionPolicy -Scope Process RemoteSigned
```

Linux / macOS :

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install fastapi uvicorn requests
```

Quand l'environnement est actif, `(.venv)` apparaît au début de la ligne de commande.

3. API de l'organisation A

Créez `api_air.py` :

```python
import sqlite3
from fastapi import FastAPI

app = FastAPI(title="API Environnement")

@app.get("/")
def accueil():
    return {"organisation": "Direction de l'environnement"}

@app.get("/mesures")
def obtenir_mesures():
    db = sqlite3.connect("environnement.db")
    db.row_factory = sqlite3.Row
    lignes = db.execute("""
        SELECT date, ville, pm25
        FROM qualite_air ORDER BY date
```

```
    """).fetchall()
    db.close()
    return [dict(ligne) for ligne in lignes]

@app.get("/statistiques")
def statistiques():
    db = sqlite3.connect("environnement.db")
    resultat = db.execute("""
        SELECT COUNT(*), AVG(pm25) FROM qualite_air
    """).fetchone()
    db.close()
    return {
        "nombre_jours": resultat[0],
        "moyenne_pm25": round(resultat[1], 2)
    }
```

Démarrez l'API dans le premier terminal :

```
python -m uvicorn api_air:app --reload --port 8001
```

Dans le navigateur, ouvrez http://127.0.0.1:8001/docs. Testez `GET /mesures` puis `GET /statistiques`.

4. API de l'organisation B

Créez `api_hopital.py` :

```
import sqlite3
from fastapi import FastAPI

app = FastAPI(title="API Hôpital")

@app.get("/")
def accueil():
    return {"organisation": "Hôpital régional"}

@app.get("/admissions")
def obtenir_admissions():
    db = sqlite3.connect("hopital.db")
    db.row_factory = sqlite3.Row
    lignes = db.execute("""
        SELECT date, ville, nombre
        FROM admissions ORDER BY date
    """).fetchall()
    db.close()
    return [dict(ligne) for ligne in lignes]

@app.get("/statistiques")
def statistiques():
    db = sqlite3.connect("hopital.db")
```

```
    resultat = db.execute("""
        SELECT COUNT(*), AVG(nombre) FROM admissions
    """).fetchone()
    db.close()
    return {
        "nombre_jours": resultat[0],
        "moyenne_admissions": round(resultat[1], 2)
    }
```

Ouvrez un deuxième terminal, réactivez l'environnement (dans le dossier du projet), puis lancez :

```
python -m uvicorn api_hopital:app --reload --port 8002
```

Consultez http://127.0.0.1:8002/docs et testez `GET /admissions` puis `GET /statistiques`. Gardez les deux serveurs en fonctionnement.

5. Client du laboratoire

Créez `client_analyse.py`. Ce client interroge les deux API, associe les résultats par date et compare les admissions.

```
import requests

URL_AIR = "http://127.0.0.1:8001"
URL_HOPITAL = "http://127.0.0.1:8002"

r_air = requests.get(f"{URL_AIR}/mesures", timeout=5)
r_air.raise_for_status()
mesures_air = r_air.json()

r_hopital = requests.get(
    f"{URL_HOPITAL}/admissions", timeout=5
)
r_hopital.raise_for_status()
admissions = r_hopital.json()

air_par_date = {x["date"]: x["pm25"] for x in mesures_air}
hopital_par_date = {x["date"]: x["nombre"] for x in admissions}

elevees = []
non_elevees = []

print("Date | PM2.5 | Admissions")
for date in sorted(air_par_date.keys() & hopital_par_date.keys()):
    pm25 = air_par_date[date]
    nombre = hopital_par_date[date]
    print(date, "|", pm25, "|", nombre)
    if pm25 > 30:
        elevees.append(nombre)
```

```
    else:
        non_elevees.append(nombre)

print("Moyenne admissions, pollution élevée :",
      round(sum(elevees) / len(elevees), 2))
print("Moyenne admissions, autres jours :",
      round(sum(non_elevees) / len(non_elevees), 2))
```

Dans un troisième terminal (environnement réactivé), exécutez :

```
python client_analyse.py
```

6. Résultat attendu

Le client affiche les cinq dates et calcule une moyenne de 16 admissions pendant les trois journées où PM2.5 > 30, contre 8,5 admissions pendant les deux autres journées.

7. Questions TP 2

1. Quel est le rôle de FastAPI ?
2. Quel est le rôle de Uvicorn ?
3. Pourquoi utilise-t-on deux ports différents ?
4. Le laboratoire reçoit-il les données brutes dans cette version ?
5. Une API REST garantit-elle automatiquement la confidentialité ?

TP 3 — Calculs locaux, contrôle d'accès et analyse fédérée

1. Objectifs

- Calculer des résultats près des données d'origine.
- Ajouter un contrôle d'accès par clé API de démonstration.
- Transmettre une catégorie de pollution plutôt que la concentration exacte.
- Faire calculer à l'hôpital les statistiques sur ses admissions locales.
- Discuter des limites de confidentialité et de souveraineté.

2. Préparation

Prérequis :

- TP 1 et TP 2 exécutés ;
- fichiers `environnement.db` et `hopital.db` présents dans le dossier.

Avant de commencer :

- arrêtez les anciennes API avec Ctrl+C ;
- vérifiez que l'environnement virtuel est actif et que les bibliothèques sont installées.

3. API protégée de l'organisation A

Créez `api_air_v3.py` :

```
import os
import sqlite3
from fastapi import FastAPI, Header, HTTPException

app = FastAPI(title="API Environnement - TP3")
CLE_API = os.getenv("CLE_ENVIRONNEMENT", "demo-air-123")

def verifier_cle(x_api_key):
    if x_api_key != CLE_API:
        raise HTTPException(status_code=401, detail="Clé API invalide")

@app.get("/")
def accueil():
    return {"organisation": "Direction de l'environnement"}

@app.get("/categories")
def categories(x_api_key: str = Header(default="")):
    verifier_cle(x_api_key)
    db = sqlite3.connect("environnement.db")
    lignes = db.execute("""
        SELECT date,
          CASE WHEN pm25 > 30 THEN 'elevee'
               ELSE 'non_elevee' END
        FROM qualite_air ORDER BY date
    """).fetchall()
    db.close()
    return [
        {"date": date, "categorie": categorie}
        for date, categorie in lignes
    ]
```

Dans un premier terminal :

PowerShell :

```
$env:CLE_ENVIRONNEMENT="demo-air-123"
python -m uvicorn api_air_v3:app --reload --port 8001
```

Linux / macOS :

```
export CLE_ENVIRONNEMENT="demo-air-123"
python -m uvicorn api_air_v3:app --reload --port 8001
```

L'API renvoie uniquement les dates et la catégorie de pollution.

Pour tester, ouvrez http://127.0.0.1:8001/docs. La requête sans clé doit être refusée ; envoyez l'en-tête `X-API-Key` avec la clé de démonstration pour obtenir une réponse.

4. API de calcul local de l'hôpital

Créez `api_hopital_v3.py` :

```
import os
import sqlite3
from collections import defaultdict
from fastapi import FastAPI, Header, HTTPException
from pydantic import BaseModel

app = FastAPI(title="API Hôpital - TP3")
CLE_API = os.getenv("CLE_HOPITAL", "demo-hopital-456")

def verifier_cle(x_api_key):
    if x_api_key != CLE_API:
        raise HTTPException(status_code=401, detail="Clé API invalide")

class Classification(BaseModel):
    date: str
    categorie: str

class DemandeAnalyse(BaseModel):
    classifications: list[Classification]

@app.get("/")
def accueil():
    return {"organisation": "Hôpital régional"}

@app.post("/analyse-locale")
def analyse_locale(
    demande: DemandeAnalyse,
    x_api_key: str = Header(default="")
):
    verifier_cle(x_api_key)
    autorisees = {"elevee", "non_elevee"}
    if any(c.categorie not in autorisees
           for c in demande.classifications):
        raise HTTPException(status_code=400, detail="Catégorie invalide")

    dates = [c.date for c in demande.classifications]
    if len(dates) != len(set(dates)):
        raise HTTPException(status_code=400, detail="Dates en double")

    groupes = {c.date: c.categorie for c in demande.classifications}
    db = sqlite3.connect("hopital.db")
    lignes = db.execute("SELECT date, nombre FROM admissions").fetchall()
    db.close()

    statistiques = defaultdict(list)
```

```
    for date, nombre in lignes:
        categorie = groupes.get(date)
        if categorie is not None:
            statistiques[categorie].append(nombre)

    resultat = {}
    for categorie in ("elevee", "non_elevee"):
        valeurs = statistiques[categorie]
        resultat[categorie] = {
            "nombre_jours": len(valeurs),
            "moyenne_admissions": (
                round(sum(valeurs) / len(valeurs), 2)
                if valeurs else None
            )
        }
    return resultat
```

Dans un deuxième terminal (environnement réactivé) :

PowerShell :

```
$env:CLE_HOPITAL="demo-hopital-456"
python -m uvicorn api_hopital_v3:app --reload --port 8002
```

Linux / macOS :

```
export CLE_HOPITAL="demo-hopital-456"
python -m uvicorn api_hopital_v3:app --reload --port 8002
```

Gardez les deux API actives. L'hôpital reçoit les catégories et calcule les statistiques à partir de sa base locale.

5. Client du laboratoire

Créez `laboratoire_v3.py` :

```
import os
import requests

URL_AIR = "http://127.0.0.1:8001"
URL_HOPITAL = "http://127.0.0.1:8002"
CLE_AIR = os.getenv("CLE_ENVIRONNEMENT", "demo-air-123")
CLE_HOPITAL = os.getenv("CLE_HOPITAL", "demo-hopital-456")

r_air = requests.get(
    f"{URL_AIR}/categories",
    headers={"X-API-Key": CLE_AIR}, timeout=5
)
r_air.raise_for_status()
classifications = r_air.json()
```

```
r_hopital = requests.post(
    f"{URL_HOPITAL}/analyse-locale",
    headers={"X-API-Key": CLE_HOPITAL},
    json={"classifications": classifications},
    timeout=5
)
r_hopital.raise_for_status()
resultats = r_hopital.json()

elevee = resultats["elevee"]
non_elevee = resultats["non_elevee"]

print("--- Analyse régionale ---")
print("Pollution élevée :", elevee)
print("Pollution non élevée :", non_elevee)

if (elevee["moyenne_admissions"] is not None
        and non_elevee["moyenne_admissions"] is not None):
    difference = (
        elevee["moyenne_admissions"]
        - non_elevee["moyenne_admissions"]
    )
    print("Différence entre les moyennes :", round(difference, 2))
```

Dans un troisième terminal (environnement réactivé) :

PowerShell :

```
$env:CLE_ENVIRONNEMENT="demo-air-123"
$env:CLE_HOPITAL="demo-hopital-456"
python laboratoire_v3.py
```

Linux / macOS :

```
export CLE_ENVIRONNEMENT="demo-air-123"
export CLE_HOPITAL="demo-hopital-456"
python laboratoire_v3.py
```

6. Résultats attendus

Catégorie	Nombre de jours	Moyenne admissions/jour
Pollution élevée	3	16
Pollution non élevée	2	8,5

Différence entre les moyennes : 7,5 admissions/jour. Ce petit jeu de données fictives suggère une association, mais ne démontre ni causalité ni relation médicale générale.

7. Tests du contrôle d'accès

Test	Manipulation	Résultat attendu
1	Appeler `/categories` sans clé	401 — accès refusé

2	Utiliser une clé incorrecte	401 — accès refusé
3	Utiliser la bonne clé	200 — réponse autorisée
4	Appeler `/analyse-locale` avec une clé incorrecte	401 — accès refusé
5	Envoyer une catégorie inconnue	400 — requête refusée

8. Mise à l'épreuve : l'attaque par différence

Le TP 3 protège l'accès par une clé, mais **ne contrôle pas ce que le laboratoire demande**.

Créez `attaque_difference.py` :

```
import requests

URL_HOPITAL = "http://127.0.0.1:8002"
CLE_HOPITAL = "demo-hopital-456"

# Le laboratoire choisit LUI-MÊME le groupe : un seul jour.
demande = {"classifications": [{"date": "2026-10-01", "categorie":
"non_elevee"}]}

r = requests.post(
    f"{URL_HOPITAL}/analyse-locale",
    headers={"X-API-Key": CLE_HOPITAL},
    json=demande,
    timeout=5,
)
print(r.json())
```

Avec le serveur de l'hôpital actif, exécutez : `python attaque_difference.py`. Résultat :

```
{'elevee': {'nombre_jours': 0, 'moyenne_admissions': None},
 'non_elevee': {'nombre_jours': 1, 'moyenne_admissions': 8.0}}
```

9. Questions TP 3

1. Quelles données le laboratoire reçoit-il dans le TP 2 ? Et dans le TP 3 ?
2. Pourquoi l'hôpital effectue-t-il les calculs dans son propre environnement ?
3. Quelles informations sont encore transmises à l'hôpital ?
4. Une statistique agrégée garantit-elle toujours la confidentialité ? (Appuyez-vous sur la section 8.)
5. Une clé API suffit-elle pour garantir la souveraineté des données ?
6. Comment limiter les demandes d'analyse aux opérations autorisées ?
7. Quels composants supplémentaires faut-il pour un véritable espace de données ?

10. Limites et sécurité

Les clés par défaut sont destinées uniquement à la démonstration. Ce prototype ne doit pas être exposé sur Internet et ne doit pas être utilisé avec des données réelles. Un système réel

nécessite HTTPS, une gestion sécurisée des secrets, des identités organisationnelles, des autorisations précises, des journaux d'accès, une gouvernance et une analyse des risques de divulgation. Les catégories quotidiennes envoyées à l'hôpital restent des informations échangées ; le calcul est local côté hôpital, mais l'ensemble n'est pas une solution de confidentialité formelle.

TP 4 — Gouvernance et confidentialité : contrat, signature et bruit

1. Objectifs

- Découvrir un jeu de données grâce à un catalogue simplifié (inspiré de DCAT).
- Contrôler l'accès selon la **finalité** d'utilisation (règle inspirée d'ODRL) et créer un contrat.
- Faire **signer** les classifications par l'organisation A pour que l'hôpital puisse les vérifier.
- Appliquer un **seuil minimal** de publication et un **bruit de Laplace** sur les résultats.
- Discuter les limites de ces protections.

2. Contexte, problème et solution

Contexte. Comme dans les TP précédents, le laboratoire veut croiser les PM2.5 (organisation A) et les admissions (organisation B), qu'il ne possède pas.

Problème. Le TP 3 laisse trois failles :

- le laboratoire peut choisir librement ses groupes de dates et retrouver une valeur individuelle (section 8 du TP 3) ;
- l'hôpital n'exige qu'une clé API : il ne sait pas si le laboratoire a obtenu un accès pour une finalité légitime ;
- le laboratoire ne dispose d'aucun moyen de découvrir les jeux de données (catalogue).

La question à laquelle répond ce TP est la suivante :

> **Comment analyser ensemble des données détenues par des organisations différentes, sans les centraliser, en contrôlant leur usage et en limitant ce qui est révélé ?**

Solution. Le nœud A publie un catalogue, vérifie la finalité, crée un contrat et **signe** la liste des classifications. Le nœud B refuse toute liste dont la signature est invalide, ne publie pas les groupes de moins de 3 jours et peut ajouter du bruit. Le laboratoire coordonne le tout sans jamais ouvrir les bases.

Déroulement :

1. C lit le catalogue de A (`GET /catalog`).
2. C demande un contrat à A (`POST /contracts/negotiate`) avec la finalité `research`.

3. A vérifie clé API, jeu de données, finalité et consommateur, puis crée le contrat.
4. C récupère les classifications (`GET /classifications`), signées (HMAC) par A pour ce contrat et ce consommateur.
5. C transmet le paquet signé à B (`POST /compute`). B vérifie la signature, calcule localement et renvoie uniquement des agrégats.

La fédération ne signifie pas qu'aucune donnée n'est échangée : les classifications et les agrégats circulent.

3. Préparation

- Créez un **nouveau dossier** `TP4_Gouvernance` et ouvrez-le dans VS Code (les bases du TP 4 ont 10 jours : elles ne doivent pas écraser celles des TP 1 à 3).
- Arrêtez les serveurs du TP 3 avec Ctrl+C (les ports 8001 et 8002 seront réutilisés).
- Créez un environnement virtuel et installez les bibliothèques (voir TP 2, section 2).
- Les clés et le secret de ce TP sont des valeurs de démonstration différentes de celles du TP 3.
- **N'utilisez pas `--reload`** : chaque modification redémarre le serveur et efface les contrats stockés en mémoire.
- Dans **chacun des trois terminaux**, placez-vous dans le dossier du TP et réactivez l'environnement virtuel.

Vous allez créer quatre fichiers : `setup_data.py`, `api_air.py`, `api_hopital.py`, `client_laboratoire.py`.

4. Étape 0 — Bases de données (10 jours)

Créez `setup_data.py` :

```python
"""Étape 0 - Création des deux bases SQLite indépendantes (données fictives)."""
from contextlib import closing
from pathlib import Path
import sqlite3

DOSSIER = Path(__file__).resolve().parent
DB_AIR = DOSSIER / "environnement.db"
DB_HOPITAL = DOSSIER / "hopital.db"

donnees_air = [
    ("2026-10-01", "Tanger", 12.0),
    ("2026-10-02", "Tanger", 35.5),
    ("2026-10-03", "Tanger", 42.1),
    ("2026-10-04", "Tanger", 15.2),
    ("2026-10-05", "Tanger", 38.0),
    ("2026-10-06", "Tanger", 45.0),
    ("2026-10-07", "Tanger", 22.1),
    ("2026-10-08", "Tanger", 18.0),
    ("2026-10-09", "Tanger", 31.0),
```

```
    ("2026-10-10", "Tanger", 11.5),
]

donnees_hopital = [
    ("2026-10-01", "Tanger", 8),
    ("2026-10-02", "Tanger", 14),
    ("2026-10-03", "Tanger", 18),
    ("2026-10-04", "Tanger", 9),
    ("2026-10-05", "Tanger", 16),
    ("2026-10-06", "Tanger", 20),
    ("2026-10-07", "Tanger", 10),
    ("2026-10-08", "Tanger", 7),
    ("2026-10-09", "Tanger", 13),
    ("2026-10-10", "Tanger", 5),
]

# closing() ferme réellement la connexion ; "with conn" seul ne fait que
valider
# la transaction (problème de verrouillage de fichier sous Windows).

# Base de l'organisation Environnement
with closing(sqlite3.connect(DB_AIR)) as conn:
    with conn:
        conn.execute("""
            CREATE TABLE IF NOT EXISTS qualite_air (
                date TEXT PRIMARY KEY,
                ville TEXT NOT NULL,
                pm25 REAL NOT NULL CHECK(pm25 >= 0)
            )
        """)
        conn.execute("DELETE FROM qualite_air")
        conn.executemany(
            "INSERT INTO qualite_air (date, ville, pm25) VALUES (?, ?, ?)",
            donnees_air,
        )

# Base de l'organisation Hôpital
with closing(sqlite3.connect(DB_HOPITAL)) as conn:
    with conn:
        conn.execute("""
            CREATE TABLE IF NOT EXISTS admissions (
                date TEXT PRIMARY KEY,
                ville TEXT NOT NULL,
                nombre INTEGER NOT NULL CHECK(nombre >= 0)
            )
        """)
        conn.execute("DELETE FROM admissions")
        conn.executemany(
            "INSERT INTO admissions (date, ville, nombre) VALUES (?, ?, ?)",
            donnees_hopital,
```

```
        )

print("Bases créées et données réinitialisées.")
```

Exécutez `python setup_data.py`. Résultat attendu : « Bases créées et données réinitialisées. » Les mesures et les admissions sont dans deux bases séparées, `environnement.db` et `hopital.db`.

5. Étape 1 — Le nœud A (Environnement)

Créez `api_air.py` :

```
"""Étape 1 - Nœud A (Environnement) : catalogue, contrat, classification
signée."""
from contextlib import closing
from pathlib import Path
import hashlib
import hmac
import json
import os
import sqlite3
import uuid

from fastapi import FastAPI, Header, HTTPException
from pydantic import BaseModel

app = FastAPI(title="Nœud A - Environnement")

DOSSIER = Path(__file__).resolve().parent
DB_AIR = DOSSIER / "environnement.db"

# Valeurs de démonstration : à ne JAMAIS utiliser en production.
CLE_API_A = os.getenv("CLE_AIR", "secret-air-key-demo")
SECRET_FEDERATION = os.getenv("SECRET_FEDERATION", "secret-federation-
demo").encode()

DATASET_ID = "dataset-air-tanger"
FINALITE_AUTORISEE = "research"
CONSOMMATEURS_AUTORISES = {"laboratoire-iasd"}
SEUIL_PM25 = 30.0  # seuil pédagogique (µg/m³), pas une référence
réglementaire

# Catalogue simplifié inspiré de DCAT (non conforme au standard).
CATALOGUE = [
    {
        "dataset_id": DATASET_ID,
        "title": "Qualité de l'air - PM2.5 à Tanger",
        "description": "Concentrations journalières de PM2.5 (données
fictives).",
        "publisher": "Organisation Environnement",
```

```
        "temporal_coverage": "2026-10-01/2026-10-10",
        "unit": "µg/m³",
        "access_endpoint": "/contracts/negotiate",
        "allowed_purpose": FINALITE_AUTORISEE,  # règle inspirée d'ODRL
    }
]

# Contrats en mémoire : perdus au redémarrage du serveur.
contrats = {}


class DemandeContrat(BaseModel):
    dataset_id: str
    consumer_id: str
    purpose: str


def verifier_cle(x_api_key):
    if x_api_key is None or not hmac.compare_digest(x_api_key, CLE_API_A):
        raise HTTPException(status_code=401, detail="Clé API invalide")


def signer(contract_id, consumer_id, classifications):
    """Signature HMAC partagée avec le nœud B : prouve que A a émis ces
    classifications pour ce contrat et ce consommateur."""
    message = json.dumps(
        {
            "contract_id": contract_id,
            "consumer_id": consumer_id,
            "classifications": classifications,
        },
        sort_keys=True,
        separators=(",", ":"),
        ensure_ascii=False,
    ).encode("utf-8")
    return hmac.new(SECRET_FEDERATION, message, hashlib.sha256).hexdigest()


@app.get("/catalog")
def lire_catalogue():
    return CATALOGUE


@app.post("/contracts/negotiate")
def negocier_contrat(demande: DemandeContrat, x_api_key: str | None =
Header(default=None)):
    verifier_cle(x_api_key)

    if demande.dataset_id != DATASET_ID:
```

```
        raise HTTPException(status_code=404, detail="Jeu de données
inexistant")

    if demande.purpose != FINALITE_AUTORISEE:
        raise HTTPException(status_code=403, detail="Finalité non autorisée")

    if demande.consumer_id not in CONSOMMATEURS_AUTORISES:
        raise HTTPException(status_code=403, detail="Consommateur non
autorisé")

    contract_id = f"contract-air-{uuid.uuid4().hex[:8]}"
    contrats[contract_id] = {
        "status": "AGREED",
        "dataset_id": demande.dataset_id,
        "consumer_id": demande.consumer_id,
        "purpose": demande.purpose,
    }
    return {"contract_id": contract_id, "status": "AGREED"}


@app.get("/classifications")
def lire_classifications(
    contract_id: str,
    consumer_id: str,
    x_api_key: str | None = Header(default=None),
):
    verifier_cle(x_api_key)

    contrat = contrats.get(contract_id)
    if contrat is None:
        raise HTTPException(status_code=403, detail="Contrat inconnu")

    if contrat["status"] != "AGREED" or contrat["dataset_id"] != DATASET_ID:
        raise HTTPException(status_code=403, detail="Contrat non applicable")

    if contrat["purpose"] != FINALITE_AUTORISEE or contrat["consumer_id"] !=
consumer_id:
        raise HTTPException(status_code=403, detail="Consommateur ou finalité
non autorisé")

    with closing(sqlite3.connect(DB_AIR)) as conn:
        lignes = conn.execute(
            "SELECT date, pm25 FROM qualite_air ORDER BY date"
        ).fetchall()

    classifications = [
        {
            "date": date,
            "categorie": "au_dessus_seuil" if pm25 > SEUIL_PM25 else
```

```
"sous_seuil",
        }
        for date, pm25 in lignes
    ]
    return {
        "contract_id": contract_id,
        "consumer_id": consumer_id,
        "classifications": classifications,
        "signature": signer(contract_id, consumer_id, classifications),
    }
```

Le nœud A applique quatre contrôles avant de créer un contrat, dans cet ordre : clé API (401), jeu de données existant (404), finalité `research` (403), consommateur autorisé (403). Il classe ensuite chaque journée (`au_dessus_seuil` si PM2.5 > 30 µg/m^3, `sous_seuil` sinon) et **signe** la liste (HMAC).

Lancez-le dans le premier terminal :

```
python -m uvicorn api_air:app --port 8001
```

Ouvrez http://127.0.0.1:8001/docs (clé `x-api-key` : `secret-air-key-demo`) et testez :

Test	Manipulation	Résultat attendu
Catalogue	`GET /catalog`	200
Accès autorisé	`POST /contracts/negotiate` avec `purpose = research`	200 et un `contract_id`
Finalité non autorisée	`purpose = commercial`	403
Jeu de données inexistant	`dataset_id` modifié	404
Clé API incorrecte	`x-api-key` modifiée	401

Corps de la requête de négociation :

```
{
  "dataset_id": "dataset-air-tanger",
  "consumer_id": "laboratoire-iasd",
  "purpose": "research"
}
```

Avec le `contract_id` obtenu, appelez `GET /classifications` (`consumer_id = laboratoire-iasd`) : vous obtenez 10 journées classées et une signature.

6. Étape 2 — Le nœud B (Hôpital)

Créez `api_hopital.py` :

```
"""Étape 2 - Nœud B (Hôpital) : calcul local d'agrégats, seuil minimal, bruit de Laplace."""
```

```python
from contextlib import closing
from pathlib import Path
import hashlib
import hmac
import json
import math
import os
import random
import sqlite3

from fastapi import FastAPI, Header, HTTPException
from pydantic import BaseModel, Field

app = FastAPI(title="Nœud B - Hôpital")

DOSSIER = Path(__file__).resolve().parent
DB_HOPITAL = DOSSIER / "hopital.db"

CLE_API_B = os.getenv("CLE_HOPITAL", "secret-hospital-key-demo")
SECRET_FEDERATION = os.getenv("SECRET_FEDERATION", "secret-federation-
demo").encode()

CATEGORIES_VALIDES = {"au_dessus_seuil", "sous_seuil"}
SEUIL_MIN_JOURS = 3      # seuil minimal de publication (PAS du k-anonymat)
BORNE_MAX = 30.0         # borne pédagogique d'admissions par jour
BUDGET_EPSILON = 3.0     # budget cumulé par contrat (simulation)

RNG = random.SystemRandom()   # source d'aléa non prévisible (et non
random.random)
budget_utilise = {}           # contract_id -> epsilon consommé


class Classification(BaseModel):
    date: str
    categorie: str


class RequeteCalcul(BaseModel):
    contract_id: str
    consumer_id: str
    classifications: list[Classification]
    signature: str
    apply_dp: bool = False
    epsilon: float = Field(default=1.0, gt=0)


def verifier_cle(x_api_key):
    if x_api_key is None or not hmac.compare_digest(x_api_key, CLE_API_B):
        raise HTTPException(status_code=401, detail="Clé API invalide")
```

```
def signature_attendue(req):
    message = json.dumps(
        {
            "contract_id": req.contract_id,
            "consumer_id": req.consumer_id,
            "classifications": [c.model_dump() for c in req.classifications],
        },
        sort_keys=True,
        separators=(",", ":"),
        ensure_ascii=False,
    ).encode("utf-8")
    return hmac.new(SECRET_FEDERATION, message, hashlib.sha256).hexdigest()


def bruit_laplace(echelle):
    u = RNG.random() - 0.5
    while u == -0.5:          # évite log(0)
        u = RNG.random() - 0.5
    return -echelle * math.copysign(1.0, u) * math.log(1 - 2 * abs(u))


def resumer(valeurs, apply_dp, epsilon):
    n = len(valeurs)
    if n < SEUIL_MIN_JOURS:
        return {"n": n, "moyenne": None, "statut":
"SUPPRIME_GROUPE_TROP_PETIT"}

    moyenne = sum(valeurs) / n
    if apply_dp:
        sensibilite = BORNE_MAX / n          # moyenne d'un groupe de taille
n fixée
        moyenne += bruit_laplace(sensibilite / epsilon)
        moyenne = min(max(moyenne, 0.0), BORNE_MAX)   # post-traitement
    return {"n": n, "moyenne": round(moyenne, 2), "statut": "OK"}


@app.post("/compute")
def calculer(req: RequeteCalcul, x_api_key: str | None =
Header(default=None)):
    verifier_cle(x_api_key)

    # Gouvernance de bout en bout : seules les classifications émises par A
    # pour un contrat valide (signature) sont acceptées.
    if not hmac.compare_digest(req.signature, signature_attendue(req)):
        raise HTTPException(status_code=403, detail="Signature invalide :
classification non émise par le nœud A")
```

```
    if not req.classifications:
        raise HTTPException(status_code=400, detail="Liste de classifications
vide")

    dates = [c.date for c in req.classifications]
    if len(dates) != len(set(dates)):
        raise HTTPException(status_code=400, detail="Date dupliquée")
    for c in req.classifications:
        if c.categorie not in CATEGORIES_VALIDES:
            raise HTTPException(status_code=400, detail=f"Catégorie inconnue
: {c.categorie}")

    with closing(sqlite3.connect(DB_HOPITAL)) as conn:
        admissions = dict(conn.execute("SELECT date, nombre FROM
admissions").fetchall())

    inconnues = [d for d in dates if d not in admissions]
    if inconnues:
        raise HTTPException(status_code=400, detail=f"Dates absentes de la
base hospitalière : {inconnues}")

    if req.apply_dp:
        utilise = budget_utilise.get(req.contract_id, 0.0)
        if utilise + req.epsilon > BUDGET_EPSILON:
            raise HTTPException(status_code=429, detail="Budget de
confidentialité épuisé pour ce contrat")
        budget_utilise[req.contract_id] = utilise + req.epsilon

    groupes = {cat: [] for cat in CATEGORIES_VALIDES}
    for c in req.classifications:
        groupes[c.categorie].append(admissions[c.date])

    return {
        "au_dessus_seuil": resumer(groupes["au_dessus_seuil"], req.apply_dp,
req.epsilon),
        "sous_seuil": resumer(groupes["sous_seuil"], req.apply_dp,
req.epsilon),
        "bruit_applique": req.apply_dp,
        "epsilon": req.epsilon if req.apply_dp else None,
    }
```

Le nœud B enchaîne les contrôles suivants : clé API (401), signature valide (403), liste non vide et sans doublon, catégories connues, dates présentes dans la base (400), budget de confidentialité si bruit demandé (429). Il calcule ensuite les moyennes localement.

- **Signature.** Sans elle, le laboratoire pourrait envoyer n'importe quel groupe de dates (attaque du TP 3). Avec elle, il ne peut utiliser que la liste complète émise par A pour son contrat.

- **Seuil minimal.** Un groupe de moins de 3 jours n'est pas publié. Ce n'est pas du k-anonymat.
- **Bruit de Laplace.** Avec `apply_dp` = `true`, l'échelle du bruit vaut `(30 / n) / ε` (30 est la borne pédagogique d'admissions par jour). Pour n = 5 et ε = 1, l'écart-type est d'environ 8,5, comparable à la différence observée : les résultats bruités sont très variables. Ce n'est **pas** une garantie formelle de confidentialité différentielle.

Lancez-le dans le deuxième terminal :

```
python -m uvicorn api_hopital:app --port 8002
```

Tester `POST /compute` dans `/docs` (http://127.0.0.1:8002/docs) :

1. Obtenez un contrat puis exécutez `GET /classifications` sur le nœud A.
2. Copiez la réponse complète (`contract_id`, `consumer_id`, `classifications`, `signature`).
3. Dans `POST /compute`, renseignez `x-api-key` avec `secret-hospital-key-demo` et collez la réponse comme corps.
4. Pour tester le bruit, ajoutez `"apply_dp": true` (et éventuellement `"epsilon": 1.0`).

Test	Manipulation	Résultat attendu
Calcul sans bruit	Corps copié tel quel	200, moyennes 16,2 et 7,8
Clé API incorrecte	`x-api-key` modifiée	401
Signature falsifiée	Supprimer une ligne de `classifications`	403
Calcul avec bruit	Ajouter `"apply_dp": true`	200, moyennes différentes à chaque appel
Budget épuisé	4e appel avec bruit et ε = 1	429

Refaites l'attaque du TP 3 : envoyer un sous-ensemble de dates est maintenant refusé (403).

7. Étape 3 — Le client laboratoire

Créez `client_laboratoire.py` :

```python
"""Étape 3 - Nœud C (Laboratoire) : orchestrateur, sans accès direct aux fichiers SQLite."""
import sys
import requests

URL_AIR = "http://127.0.0.1:8001"
URL_HOPITAL = "http://127.0.0.1:8002"
CLE_A = "secret-air-key-demo"
CLE_B = "secret-hospital-key-demo"
CONSUMER_ID = "laboratoire-iasd"
DATASET_ID = "dataset-air-tanger"
```

```
TIMEOUT = 10


def negocier(session, purpose, cle=CLE_A, dataset_id=DATASET_ID):
    return session.post(
        f"{URL_AIR}/contracts/negotiate",
        json={"dataset_id": dataset_id, "consumer_id": CONSUMER_ID,
"purpose": purpose},
        headers={"X-API-Key": cle},
        timeout=TIMEOUT,
    )


def executer_scenario(apply_noise=False, epsilon=1.0):
    session = requests.Session()

    print("1. Découverte du catalogue")
    r = session.get(f"{URL_AIR}/catalog", timeout=TIMEOUT)
    r.raise_for_status()
    print("  ", r.json()[0]["title"])

    print("\n2. Tests négatifs")
    r = negocier(session, "commercial")
    print(f"   Finalité commerciale : attendu 403, reçu {r.status_code}")
    r = negocier(session, "research", dataset_id="dataset-inconnu")
    print(f"   Jeu de données inexistant : attendu 404, reçu
{r.status_code}")
    r = negocier(session, "research", cle="mauvaise-cle")
    print(f"   Clé API incorrecte : attendu 401, reçu {r.status_code}")

    print("\n3. Demande autorisée pour la recherche")
    r = negocier(session, "research")
    r.raise_for_status()
    contract_id = r.json()["contract_id"]
    print("   Accord obtenu :", contract_id)

    print("\n4. Récupération des classifications")
    r = session.get(
        f"{URL_AIR}/classifications",
        params={"contract_id": contract_id, "consumer_id": CONSUMER_ID},
        headers={"X-API-Key": CLE_A},
        timeout=TIMEOUT,
    )
    r.raise_for_status()
    paquet = r.json()
    print("   Nombre de jours classifiés :", len(paquet["classifications"]))

    print("\n5. Calcul réalisé par l'hôpital", "(avec bruit)" if apply_noise
else "(sans bruit)")
```

```
    r = session.post(
        f"{URL_HOPITAL}/compute",
        json={**paquet, "apply_dp": apply_noise, "epsilon": epsilon},
        headers={"X-API-Key": CLE_B},
        timeout=TIMEOUT,
    )
    r.raise_for_status()
    res = r.json()

    for nom, cle in [("Pollution au-dessus du seuil", "au_dessus_seuil"),
                     ("Pollution sous le seuil", "sous_seuil")]:
        g = res[cle]
        print(f"   {nom} : n={g['n']}, moyenne={g['moyenne']},
statut={g['statut']}")

    a, b = res["au_dessus_seuil"]["moyenne"], res["sous_seuil"]["moyenne"]
    if a is not None and b is not None:
        print(f"   Différence : {a - b:.2f} admissions/jour")

    print("\n6. Test négatif : classifications falsifiées envoyées à
l'hôpital")
    faux = dict(paquet)
    faux["classifications"] = paquet["classifications"][:4]  # sous-ensemble
choisi par le labo
    r = session.post(f"{URL_HOPITAL}/compute", json=faux,
                     headers={"X-API-Key": CLE_B}, timeout=TIMEOUT)
    print(f"   Attendu 403 (signature invalide), reçu {r.status_code}")


if __name__ == "__main__":
    executer_scenario(apply_noise="--bruit" in sys.argv)
```

Avec les deux serveurs actifs, dans le troisième terminal :

```
python client_laboratoire.py
python client_laboratoire.py --bruit
```

8. Résultats attendus

Date	PM2.5 (µg/m³)	Admissions	Catégorie
01/10	12,0	8	sous_seuil
02/10	35,5	14	au_dessus_seuil
03/10	42,1	18	au_dessus_seuil
04/10	15,2	9	sous_seuil
05/10	38,0	16	au_dessus_seuil
06/10	45,0	20	au_dessus_seuil
07/10	22,1	10	sous_seuil

08/10	18,0	7	sous_seuil
09/10	31,0	13	au_dessus_seuil
10/10	11,5	5	sous_seuil

Calcul manuel :

- Au-dessus du seuil : (14 + 18 + 16 + 20 + 13) / 5 = 81 / 5 = **16,2**
- Sous le seuil : (8 + 9 + 10 + 7 + 5) / 5 = 39 / 5 = **7,8**
- Différence : 16,2 − 7,8 = **8,4** admissions par jour

Sortie du client sans bruit (l'identifiant du contrat varie) :

```
1. Découverte du catalogue
   Qualité de l'air - PM2.5 à Tanger

2. Tests négatifs
   Finalité commerciale : attendu 403, reçu 403
   Jeu de données inexistant : attendu 404, reçu 404
   Clé API incorrecte : attendu 401, reçu 401

3. Demande autorisée pour la recherche
   Accord obtenu : contract-air-32348a2e

4. Récupération des classifications
   Nombre de jours classifiés : 10

5. Calcul réalisé par l'hôpital (sans bruit)
   Pollution au-dessus du seuil : n=5, moyenne=16.2, statut=OK
   Pollution sous le seuil : n=5, moyenne=7.8, statut=OK
   Différence : 8.40 admissions/jour

6. Test négatif : classifications falsifiées envoyées à l'hôpital
   Attendu 403 (signature invalide), reçu 403
```

Avec bruit, les moyennes changent à chaque exécution ; les effectifs restent à 5 par groupe car ils ne sont pas bruités.

9. Questions TP 4

1. Quelles informations permettent au laboratoire de découvrir et de comprendre le jeu de données ?
2. Qu'est-ce que DCAT et pourquoi le catalogue est-il une simplification inspirée de DCAT ?
3. Pourquoi contrôler la finalité avant d'autoriser l'accès ? Différence entre un catalogue, une politique d'usage et un contrôle d'accès.
4. Quelles informations circulent entre les trois nœuds ? Que reste-t-il dans la base hospitalière ? Pourquoi la conservation locale ne suffit-elle pas à garantir la confidentialité ?

5. Pourquoi ne pas publier une statistique calculée sur un groupe trop petit ? Pourquoi un seuil de trois jours n'est-il pas une preuve de k-anonymat ? Comment la signature limite-t-elle l'attaque du TP 3 ?
6. Comparez les résultats avec et sans bruit. Expliquez le compromis précision / protection. Pourquoi ce code n'est-il pas une implémentation complète de confidentialité différentielle ?
7. La différence observée permet-elle de conclure que la pollution provoque les admissions ? Quelles variables faudrait-il considérer ?

Travail final à rendre

- Les fichiers Python des quatre TP.
- Les bases SQLite générées.
- Des captures d'écran de l'installation, des API (`/docs`) et des résultats.
- Les calculs manuels du TP 4 et au moins trois tests négatifs (finalité non autorisée, clé incorrecte, signature falsifiée).
- Un schéma d'architecture montrant les organisations, les bases, les API et le laboratoire.
- Les réponses aux questions de chaque TP.
- Une conclusion qui distingue stockage fédéré, traitement fédéré et analyse fédérée, et qui explique en quoi le TP 4 corrige les limites du TP 3.